This configuration is reminiscent of the middleware pipeline used by ASP.NET Core applications, but this is an outbound pipeline. When an `HttpClient` makes a request, each handler gets a chance to modify the request before the final `HttpClientHandler` makes the real HTTP request. Each handler in turn then gets a chance to view the response after it's received.

TIP You'll see an example of using this handler pipeline for cross-cutting concerns in section 33.3 when we add a transient error handler.

The problems of socket exhaustion and DNS I described in section 33.1 are related to the disposal of the `HttpClientHandler` at the end of the handler pipeline. By default, when you dispose of an `HttpClient`, you dispose of the handler pipeline too. `IHttpClientFactory` separates the lifetime of the `HttpClient` from the underlying `HttpClientHandler`.

Separating the lifetime of these two components enables the `IHttpClientFactory` to solve the problems of socket exhaustion and DNS rotation. It achieves this in two ways: